10-15 Minute Presentation Playbook

AI-Powered Talent Acquisition Platform

Speaker Roles: You will narrate the technical flows and design choices, while your partner operates the Candidate Portal (localhost:8501) and Recruiter Dashboard (localhost:8502) live.

Section 1: The Project Overview & Problem Statement (3 Minutes)

[ACTION]: Open the Candidate Portal home page at http://localhost:8501. Keep the landing screen visible.

[WHAT YOU SAY]:

"Good morning, respected mentors and panel members. Today, we are presenting our AI-Powered Talent Acquisition Platform. Let us begin by discussing the operational bottlenecks in recruitment. The recruitment funnel is heavily bottlenecked at the sourcing and screening stages. HR teams receive hundreds of resumes daily, making manual parsing slow and prone to subjective bias. Furthermore, candidate tracking systems traditionally rely on keyword-based lookups. This means a candidate who games the keywords gets shortlisted, while a highly qualified applicant who uses alternative terminology is ignored."

"Our platform addresses this by automating the end-to-end evaluation cycle. We have engineered a solution that uses natural language processing to extract skills semantically, matching them against job profiles using vector search. The candidate is then put through an interactive, voice-based technical interview. The system records, compresses, and transcribes candidate answers on-the-fly, culminating in an LLM-generated evaluation report. Let's walk through the live candidate experience from the beginning."

Section 2: Resume Parsing & Semantic Matching (2 Minutes)

[ACTION]: Partner uploads a PDF resume and clicks "Process & Match Roles".

[WHAT YOU SAY]:

"Now, my partner has uploaded a PDF resume. In the backend, our application reads the PDF binary and extracts the raw text. To extract structured information such as the candidate's name, email, phone, location, and key skills, we utilize a library called Instructor. Instructor wraps around our LLM API queries, forcing the LLM to structure its output according to a strict Pydantic schema. This ensures we get clean, reliable JSON data without any parser errors."

"Once skills are extracted, the system calculates semantic matching. Instead of checking if exact words are present, we convert the candidate's resume profile and our database job roles into high-dimensional vector embeddings using a BERT-based transformer model. We run a Cosine Similarity calculation between the candidate's embedding and the job role embeddings. The app then ranks the jobs and displays the Top 5 recommended roles with their match percentages. My partner will select the Python Developer role and apply."

Section 3: The AI Technical Interview & Voice Pipeline (3 Minutes)

[ACTION]: Partner clicks "Apply & Start Interview". A question appears. Partner clicks "Start Recording", answers the question, clicks "Stop & Transcribe", then clicks "Submit Answer". Repeat for one or two questions.

[WHAT YOU SAY]:

"Once the candidate applies, the system initializes a new document in MongoDB and generates the first interview question. To ensure the interview is rigorous, the LLM generates 5 dynamic questions customized to the job role. The candidate can read the question and click record. When the candidate stops recording, the app transcribes their response in real-time using the Google Speech-to-Text API."

"Under the hood, we solved a major database engineering problem regarding voice storage. Storing raw WAV recordings would quickly exhaust our database storage. To resolve this, our backend downsamples the audio and compresses it into the MP3 format in real-time using LAME encoder bindings (lameenc). This reduces our audio files by over 80%, from 15MB down to less than 400KB. The compressed MP3 binary is then encoded into a Base64 string and stored directly inside the candidate's MongoDB document. This stateless database design allows us to easily scale the web servers."

"Furthermore, we have engineered a high-availability fallback. If the LLM server goes offline or has connection issues during the interview, the app automatically switches to a pre-defined queue of 5 distinct, role-specific professional questions. This ensures the candidate has a smooth, uninterrupted interview even in network failures."

Section 4: Job Preferences & AI Evaluation Generation (3 Minutes)

[ACTION]: After the 5th question is answered, the screen changes. Partner fills out the Job Preferences form (Expected Salary, Work Mode, Relocation choice, and Current Location) and clicks "Submit Application & Generate Report". Wait for the evaluation to load on screen.

[WHAT YOU SAY]:

"After the candidate answers all five questions, they are directed to the final Candidate Preferences page. Here, they input operational preferences such as their target salary expectations, relocation availability, and preferred working model—whether they prefer Remote, Hybrid, or On-Site work. Simultaneously, we fetch browser coordinates using the streamlit-geolocation package, resolving them to an address via OpenStreetMap's Nominatim API, with a backup IP Geolocation fallback to ensure the user's location is recorded automatically."

"When the candidate clicks 'Submit Application & Generate Report', the candidate portal triggers our background evaluator. The app compiles the candidate's resume text, job preferences, and the full Q&A; transcript into an evaluation prompt. This prompt is dispatched to our remote LLM server with a robust 90-second query timeout to guarantee generation completeness. The LLM reads the transcript, assesses candidate skills and communication, and generates a structured AI scorecard. As you can see on the candidate's screen, they are immediately shown a success confirmation along with their communication score, soft skills review, and system hiring recommendation. Let us now examine how this looks in the recruiter's system."

Section 5: Recruiter Dashboard & Analytics (3 Minutes)

[ACTION]: Switch browser tab to the Recruiter Dashboard at http://localhost:8502. Expand the "Recruitment Analytics & Insights" tab, select the candidate you just created, scroll through their scorecard, and click the play button on their audio answer. Finally, click "Export Candidate Report (TXT)".

[WHAT YOU SAY]:

"Let's switch over to the recruiter's perspective, running on port 8502. When a recruiter logs in, they are greeted by an analytics suite. The dashboard displays native Streamlit charts plotting application counts by job role and experience distributions, giving recruiters immediate high-level hiring insight. We can sort and filter candidates easily."

"Let's look at the candidate we just created. On the right, the detailed panel displays their contact details and their AI Scorecard. This scorecard is compiled at the end of the candidate application, where our LLM reads the entire interview transcript, evaluates technical depth and communication clarity, compiles summaries of strengths and gaps, and returns a system recommendation."

"Recruiters can review each question and answer transcript. Most importantly, the recruiter can click the play button and **listen to the candidate's actual voice recording** directly in the browser! The app pulls the Base64 string from MongoDB, decodes it back to MP3 bytes on-the-fly, and feeds it to the audio player. The recruiter can write notes, update the status, and download a clean copy of the entire scorecard using the Export button."

Section 6: Security Architecture & Closing (2 Minutes)

[ACTION]: Keep the recruiter dashboard visible, showing the clean design.

[WHAT YOU SAY]:

"To conclude, we would like to highlight our security architecture. Instead of running a single Streamlit app, we split the system into two isolated services. The candidate portal on port 8501 has no access to evaluation details or recruiter notes. The recruiter portal on port 8502 is restricted. This isolated design is secure, production-ready, and guarantees data privacy."

"To summarize, we have built a semantic resume parser, a real-time MP3 audio compression pipeline, an interactive voice-enabled AI technical screener, and a detailed recruitment dashboard. Thank you, and we are now open to any questions from the panel."

Expert Panel Q&A; Cheat Sheet

Q1: Why did the candidate match score show 0.0% in your initial tests?

Answer: The evaluation prompt is highly complex and requires generating a detailed scorecard. Our initial HTTP request had a short 10-second timeout. On a remote LLM server, this complex query took longer than 10 seconds, triggering a ReadTimeout. We increased the query timeout to 90 seconds, which resolved the issue and now generates evaluations successfully.

Q2: What happens if Nominatim reverse-geocoding fails during candidate registration?

Answer: We implemented an automatic fallback to an IP Geolocation API (ip-api.com). If both fail, it falls back to a manual text field, ensuring candidate signup never blocks.

Q3: What are the security benefits of separating candidate and recruiter portals into two ports?

Answer: Running them as separate apps ensures that candidate-facing code never contains recruiter logic. Even if a candidate is skilled at web inspection, they cannot bypass security layers or find routes to recruiter panels as they run on completely separate servers.